Spotify Hit Predictor: Hit Classification

Simone Zanetti

2025-02-09

Project Presentation

In this project, we address a **binary classification problem** on the *Spotify Hit Predictor Dataset* (1960-2019).

Objective To predict whether a given song becomes a **Hit** or **Not Hit** based on various audio features.
This classification helps in making informed decisions for:
- *Music marketing investments*
- *Playlist curation strategies*

Context

- *A hit song* can generate significant revenue and recognition.
- *False negatives (missed hits)* result in missed opportunities.

Focus of the Analysis

- Emphasize **Recall for the Hit Class**, ensuring that potential hits are rarely missed.
- Maintain a balance with overall classification accuracy.

Dataset Presentation and Preprocessing

The dataset contains information on 6,398 songs from the 2010s with 19 attributes. These attributes include audio features such as danceability, energy, loudness, tempo, and more, as well as metadata. The target variable, `target`, is a binary indicator (1 for Hit, 0 for Not Hit). *Note:* The term "Flop" does not imply that a track is of inferior quality; rather, it indicates that the track likely did not achieve mainstream popularity.

Originally, the dataset was composed of data sets divided by decades, namely:

- 60/70, 80/90, 90/2000, 2000/2010, **2010/2020**.

For scalability reasons, i have chosen to exclusively retain the 2010/2020 set.

Below, we load all the libraries necessary for data import, manipulation, visualization, and modeling.

```
library(tidyverse)    # Data manipulation (includes dplyr, tidyr, ggplot2, etc.)
library(ggplot2)      # Advanced plotting
library(gridExtra)    # Arranging multiple plots in a grid
library(tidymodels)   # Modeling and machine learning framework
library(leaps)        # Feature selection methods
library(glmnet)       # Regularized regression (Ridge and Lasso)
library(pROC)         # ROC curve analysis and AUC calculation
library(rsample)      # Splitting data into training and test sets
```

```r
library(correlation)  # Correlation and partial correlation analysis
library(DataExplorer) # Automated exploratory data analysis
library(knitr)        # Integration of code and output in Markdown
library(corrplot)     # Graphical representation of correlation matrices
library(regclass)     # Additional regression tools and diagnostics
library(outliers)     # Outlier detection and handling
library(dplyr)        # Explicitly loaded for convenience
library(caret)        # Machine learning, preprocessing, and tuning
library(class)        # K-Nearest Neighbors (KNN) classification
```

Data Uploading

We load the dataset from a CSV file named `"spotify_hit_predictor.csv"`.After loading, we display the first 5 rows to verify that the data was imported correctly.

```r
dataset_2010s <- read_csv("dataset-of-10s.csv")
```

```
## Rows: 6398 Columns: 19
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (3): track, artist, uri
## dbl (16): danceability, energy, key, loudness, mode, speechiness, acousticne...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(dataset_2010s)
```

```
## # A tibble: 6 x 19
##   track        artist uri   danceability energy   key loudness  mode speechiness
##   <chr>        <chr>  <chr>        <dbl>  <dbl> <dbl>    <dbl> <dbl>       <dbl>
## 1 Wild Things  Aless~ spot~        0.741  0.626     1    -4.83     0      0.0886
## 2 Surfboard    Esqui~ spot~        0.447  0.247     5   -14.7      0      0.0346
## 3 Love Someone Lukas~ spot~        0.55   0.415     9    -6.56     0      0.052
## 4 Music To My~ Keys ~ spot~        0.502  0.648     0    -5.70     0      0.0527
## 5 Juju On Tha~ Zay H~ spot~        0.807  0.887     1    -3.89     1      0.275
## 6 Here's To N~ Avril~ spot~        0.482  0.873     0    -3.14     1      0.0853
## # i 10 more variables: acousticness <dbl>, instrumentalness <dbl>,
## #   liveness <dbl>, valence <dbl>, tempo <dbl>, duration_ms <dbl>,
## #   time_signature <dbl>, chorus_hit <dbl>, sections <dbl>, target <dbl>
```

Data Preprocessing

We consider now the features described in the dataset that represent the characteristics for each unit. We searched for the meaning of each one and we report the descriptions in the following table:

Attribute	Description
track	The name of the track.
artist	The name of the artist.
uri	The Spotify resource identifier for the track.
danceability	Describes how suitable a track is for dancing, based on tempo, rhythm stability, beat strength, and overall regularity (0.0 = least danceable, 1.0 = most danceable).
energy	Represents the intensity and activity of a track (0.0 to 1.0). Energetic tracks feel fast, loud, and noisy.
key	The estimated overall key of the track using standard pitch class notation (e.g., 0 = C, 1 = C#/Db, etc.). If no key is detected, the value is -1.
loudness	The overall loudness of the track in decibels (dB). Typical values range from -60 to 0 dB.

Attribute	Description
mode	Indicates the modality (major = 1, minor = 0) of the track, determining the type of scale used in the melody.
speechiness	Measures the presence of spoken words in a track. Values above 0.66 suggest predominantly spoken content, while values below 0.33 typically represent music.
acousticness	A confidence measure (0.0 to 1.0) of whether the track is acoustic.
instrumentalness	Predicts whether a track contains vocals. Values closer to 1.0 indicate a higher likelihood that the track is instrumental.
liveness	Detects the presence of an audience in the recording. Values above 0.8 indicate a strong likelihood that the track was performed live.
valence	Describes the musical positiveness conveyed by the track (0.0 to 1.0). Tracks with high valence sound more positive, while tracks with low valence sound more negative.
tempo	The overall estimated tempo of the track in beats per minute (BPM).
duration_ms	The duration of the track in milliseconds.
time_signature	The estimated overall time signature (meter) of the track, which indicates how many beats are in each bar.
chorus_hit	The estimated start time (in milliseconds) of the track's chorus (often taken as the start of the third section).
sections	The number of sections in the track, as extracted from Spotify's audio analysis.
target	The binary target variable: '1' indicates the track is a Hit (has featured on the Billboard Hot-100 at least once in the decade), and '0' indicates it is a Flop.

Renaming Columns and Checking for Missing Values

Before proceeding with further analysis, we standardize the column names by replacing spaces with underscores and *removing any non-alphanumeric characters*. This ensures that the column names are syntactically valid and easy to reference in our code.

After renaming, we *check for any missing values* in the dataset. In our case, the output confirms that there are no missing values.

```
names(dataset_2010s) <- gsub("\\s+", "_", names(dataset_2010s))
names(dataset_2010s) <- gsub("[^[:alnum:]_]", "", names(dataset_2010s))

missing_values <- colSums(is.na(dataset_2010s))
missing_values <- missing_values[missing_values > 0]
if (length(missing_values) > 0) {
  cat(" Missing values per column: ", paste(names(missing_values), missing_values, sep=": ", collapse=", "), "\n")
} else {
  cat("No missing values detected.\n")
}
```

```
## No missing values detected.
```

Removing Unnecessary Identifier Columns and Checking for Constant Variables The columns track, artist, and uri function solely as identifiers and do not contribute predictive information. Therefore, these columns are removed from the dataset.

To identify non-informative variables, the number of unique values in each column is computed. Columns containing only a single unique value (i.e., constant across all observations) do not provide variability for predictive modeling and are eliminated to improve model efficiency.

```
dataset_cleaned <- dataset_2010s %>% select(-track, -artist, -uri)
```

```
# Check for constant columns: we calculate the number of distinct values per column.
constant_cols <- dataset_cleaned %>%
  summarise_all(n_distinct) %>%
  gather(key = "key", value = "value") %>%
  filter(value == 1) %>%
  pull(key)
```

```r
# Remove any constant columns (if any exist) from dataset_cleaned.
dataset_cleaned <- dataset_cleaned %>% select(-all_of(constant_cols))
if (length(constant_cols) > 0) {
  kable(data.frame(Constant_Columns = constant_cols), caption = "  Columns Removed")
} else {
  cat("No constant columns found.\n")
}
```

```
## No constant columns found.
```

Data transformations: The mutate() function is used to:

Create std_loudness and std_tempo by applying our standardize() function to the loudness and tempo columns. We standardized std_loudness and std_tempo, applied a log transformation to duration_ms and instrumentalness (to reduce skewness and avoid log(0)), and removed the original columns to prevent redundancy.

Converting the Target Variable: Finally, the target variable is converted to a factor, which is necessary for classification models to properly recognize the target as a categorical outcome.

```r
standardize <- function(x) {
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
}
dataset_cleaned <- dataset_cleaned %>%
  mutate(
    std_loudness = standardize(loudness),
    std_tempo = standardize(tempo),
    log_duration_ms = log(duration_ms + 1),
    log_instrumentalness = log(instrumentalness + 0.0001)
  ) %>%
  dplyr::select(-loudness, -tempo, -duration_ms, -instrumentalness)

# This is necessary for classification tasks, as it explicitly indicates that
# the target variable is categorical.
dataset_cleaned <- dataset_cleaned %>% mutate(target = as.factor(target))
```

The *target class distribution* is now analyzed to assess balance between hit and flop categories

```r
cat("Target distribution (hit/flop): ", paste(names(prop.table(table(dataset_cleaned$target))),
    round(prop.table(table(dataset_cleaned$target)), 2), sep=": ", collapse=", "), "\n")
```

```
## Target distribution (hit/flop):  0: 0.5, 1: 0.5
```

Splitting in Training e Test Sets

Before continuing to explore the properties of our data, we divide them into two different datasets: the training dataset and the test dataset that will be used respectively for training our models and for testing the models' performances. We proceed with all the considerations about the features focusing on the training one, trying to understand the relations between them. The test set will remain "unknown". For doing the split, the proportions defined are 0.75 and 0.25, respectively for the two sets.

```r
#  Train-Test Split
set.seed(0607)

split <- initial_split(dataset_cleaned, prop = 0.75)  # 75% training, 25% test
train <- training(split)
test <- testing(split)

train_dist <- prop.table(table(train$target))
test_dist <- prop.table(table(test$target))

cat("\n Proportion of hit/flop (Train):", round(train_dist[1], 3), "Hit -", round(train_dist[2], 3), "Flop\n")
```

```
##
##  Proportion of hit/flop (Train): 0.5 Hit - 0.5 Flop
```

```r
cat("\n Proportion of hit/flop (Test):", round(test_dist[1], 3), "Hit -", round(test_dist[2], 3), "Flop\n")
```

```
##
##  Proportion of hit/flop (Test): 0.499 Hit - 0.501 Flop
```

Data Analysis

The training dataset is analyzed to examine the relationships among features and their association with the response variable (Hit/Not Hit). This exploratory analysis is essential for objectively **evaluating how the available characteristics influence the outcome and guiding subsequent modeling steps**. Understanding these relationships provides valuable insights for developing more effective predictive models for classifying songs as hits or flops.

Correlation Analysis

The first step in our data analysis involves investigating the relationships between the various features and the response variable (Hit/Not Hit). Correlation is a statistical measure that quantifies the strength and direction of a linear relationship between two variables. In essence, it **allows us to observe how changes in one variable are associated with changes in another**.

While this analysis only captures linear connections between the covariates, it serves as a useful starting point for: - Identifying which features may be more strongly related to the outcome. - Understanding the potential interplay among the predictors. - Informing subsequent modeling decisions by highlighting which variables might be most relevant.

By examining these correlations, we can gain initial insights into the data structure and determine a rational basis for feature selection in our predictive modeling efforts.

```r
train$target_numeric <- as.numeric(as.character(train$target)) - 1
correlations <- train %>%
  dplyr::select(where(is.numeric)) %>%
  dplyr::select(-target_numeric) %>%
  summarise(across(everything(), ~ cor(.x, train$target_numeric, use = "complete.obs"))) %>%
  gather(key = "Variable", value = "Correlation") %>%
  arrange(desc(abs(Correlation)))
cat("Most correlated variables with target:\n")
```

```
## Most correlated variables with target:
```

```r
print(head(correlations))
```

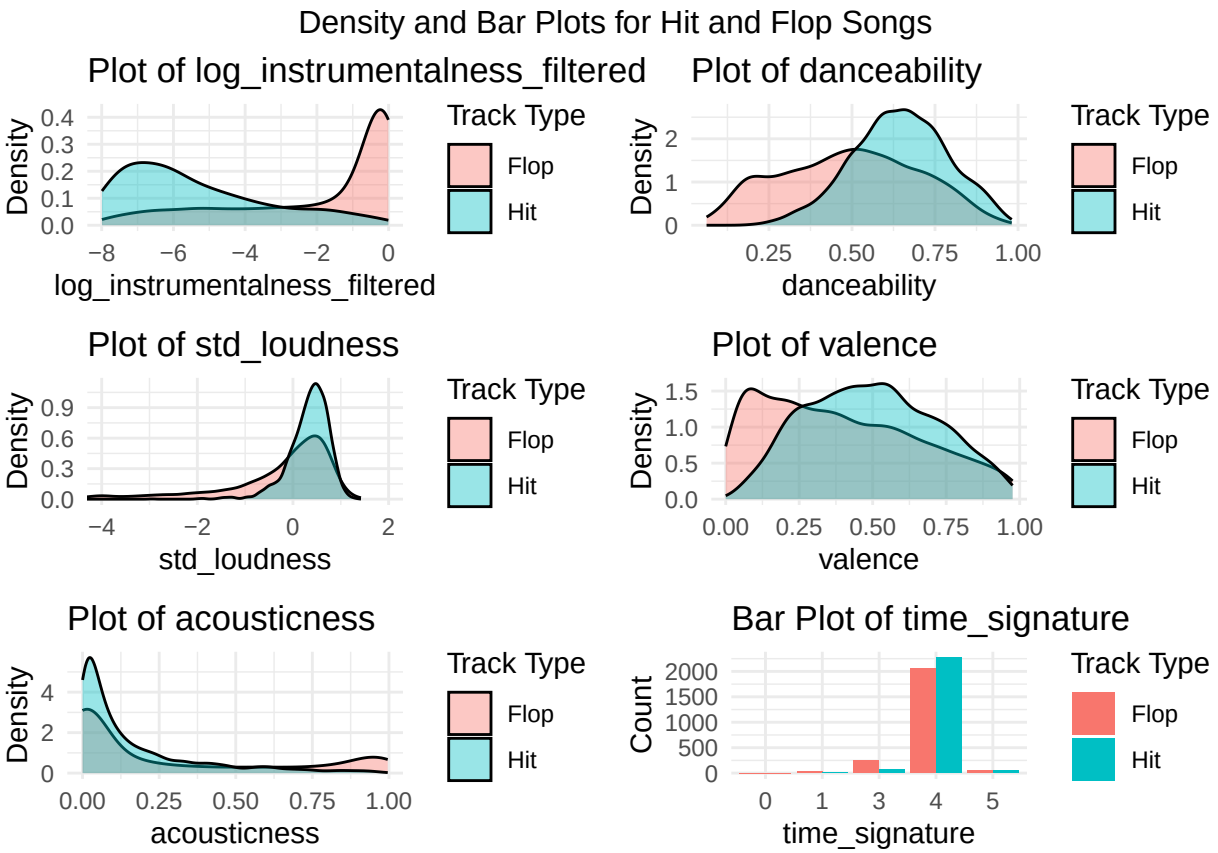
```
## # A tibble: 6 x 2
##   Variable             Correlation
##   <chr>                      <dbl>
## 1 log_instrumentalness      -0.611
## 2 danceability               0.397
## 3 std_loudness               0.331
## 4 valence                    0.208
## 5 acousticness              -0.193
## 6 time_signature             0.127
```

The table of correlations *shows how each predictor is linearly associated with our target variable* (Hit, represented as a numeric variable). Here are some *key observations* from the first results:

1. **Log Instrumentalness (-0.61)**: The negative sign indicates that *tracks with higher instrumentalness (i.e., those with fewer or no vocals) tend to be less likely to be hits.* This makes intuitive sense, as mainstream hit songs often contain prominent vocal elements.
2. **Danceability (0.39)**: A positive correlation of about 0.40 suggests that *more danceable tracks are more likely to become hits.*
3. **Loudness (0.33)**: With a correlation of around 0.33, *louder tracks tend to be associated with hits.* This may indicate that tracks with higher perceived energy and volume are more commercially successful.

```r
library(tidyverse)
library(gridExtra)
```

```r
top_variables <- c("log_instrumentalness", "danceability",
                   "std_loudness", "valence",
                   "acousticness", "time_signature")

train <- train %>%
  mutate(log_instrumentalness_filtered = ifelse(log_instrumentalness < -8, NA, log_instrumentalness))

var_limits <- list(
  log_instrumentalness_filtered = c(-8, 0),
  std_loudness = c(-4, 2)
)

plots <- lapply(top_variables, function(var) {
  if (var == "time_signature") {
    p <- ggplot(train, aes(x = as.factor(time_signature), fill = target)) +
      geom_bar(position = "dodge") +
      scale_fill_discrete(name = "Track Type", labels = c("Flop", "Hit")) +
      labs(
        title = "Bar Plot of time_signature",
        x = "time_signature",
        y = "Count"
      ) +
      theme_minimal()
  } else {
    data_to_plot <- train
    if (var == "log_instrumentalness") {
      data_to_plot <- train %>% filter(!is.na(log_instrumentalness_filtered))
      var <- "log_instrumentalness_filtered"
    }
    p <- ggplot(data_to_plot, aes(x = !!sym(var), fill = target, group = target)) +
      geom_density(alpha = 0.4, adjust = 1, na.rm = TRUE) +
      scale_fill_discrete(name = "Track Type", labels = c("Flop", "Hit")) +
      labs(
        title = paste("Plot of", var),
        x = var,
        y = "Density"
      ) +
      theme_minimal()
    if (var %in% names(var_limits)) {
      p <- p + coord_cartesian(xlim = var_limits[[var]])
    }
  }
  return(p)
})

grid.arrange(
  grobs = plots,
  ncol = 2,
  top = "Density and Bar Plots for Hit and Flop Songs"
)
```




```
train <- train %>% select(-log_instrumentalness_filtered)
cat("\\FloatBarrier")
```

Observation Based on Initial Analyses Preliminary results indicate that songs classified as hits generally exhibit higher danceability, greater loudness, and lower instrumentalness compared to flops. Specifically:

Instrumentalness and Hit Status - Correlation Summary

Feature	Correlation with Hit	Interpretation
Instrumentalness	**Negative**	*Instrumental tracks are less likely to be hits*
Danceability	**Positive**	*More rhythmic songs are more popular*
Loudness	**Positive**	*Louder, energetic songs tend to be hits*
Valence (Positivity)	**Slight Positive**	*Happier songs have a commercial edge*
Acousticness	**Negative**	*Acoustic-heavy songs are less frequent in charts*
Time Signature	**Neutral/Standard**	*Most hits follow conventional time signatures*

Where: -Negative –> Less related to Hit songs. -Positive –> More related to Hit songs.

In essence, **hit songs in this dataset appear more often to be vocal (low instrumentalness), danceable, energetic, and louder, with a more positive feel than their flop counterparts**. These findings align with general expectations of what reaches mainstream popularity—tracks that are catchy, energetic, and easy to dance to.

Focusing now on the **correlations between the covariates**, we can build the following matrix:

```
train <- train %>% select(-target_numeric)
numeric_features <- train %>% select(where(is.numeric))
```

```r
cor_matrix <- cor(numeric_features, use = "complete.obs")
corrplot(cor_matrix, method = "number", diag = FALSE, tl.cex = 0.6, number.cex = 0.6, tl.col = "black")
```

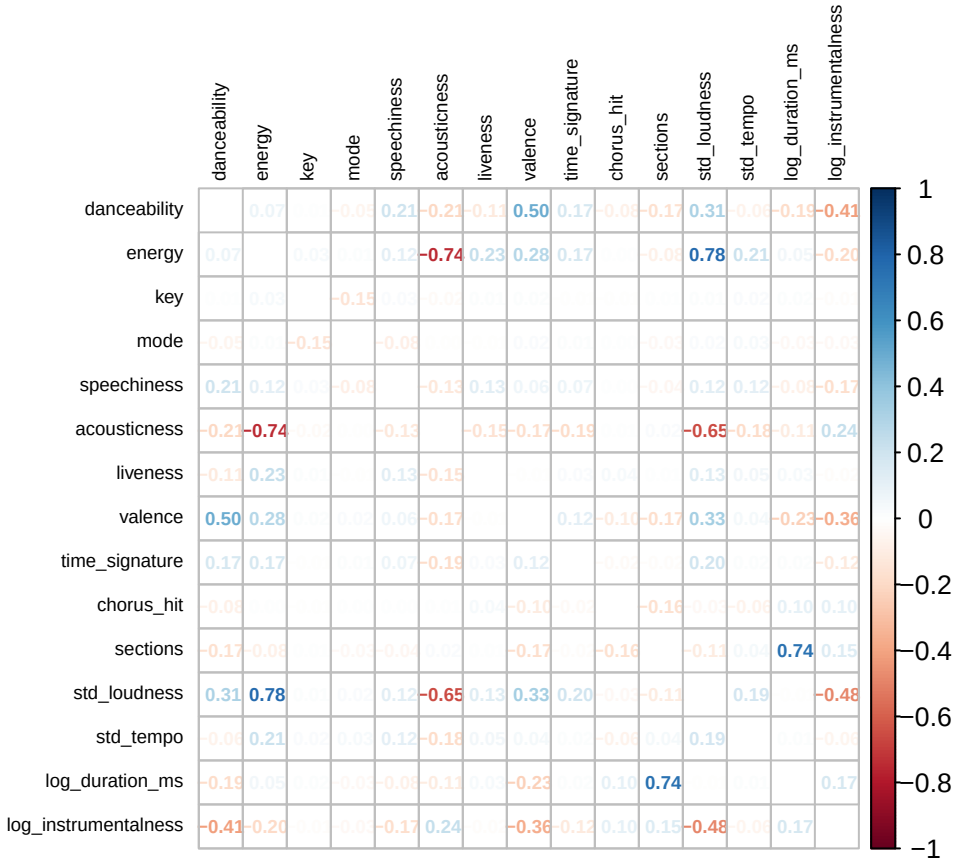


Figure 1: Figure: Correlation Matrix Between Covariates

Focus on the most notable correlations (absolute value above ~0.3), as these often represent meaningful linear relationships among features:

1. **Energy vs. std_loudness (0.78)**:*higher-energy tracks tend to be louder.* This aligns with the idea that "energetic" songs usually have higher amplitude or perceived volume.
2. **Acousticness vs. energy/std_loudness (−0.74, −0.65)**: Tracks that are *highly acoustic tend to be quieter and less energetic* (e.g., unplugged/acoustic styles vs. high-energy electronic tracks).
3. **Danceability vs. valence (0.50)**: A moderate positive correlation suggests that *more danceable songs also tend to sound "happier" or more positive.* This makes sense if upbeat rhythms often accompany cheerful or major-key music.
4. **time_signature's generally weak correlation with most variables**: Most popular music has time signatures near 4/4, so the correlation with other features remains quite low.
5. **log_instrumentalness vs. danceability/acousticness (−0.4, 0.24)**: After log-transforming instrumentalness, it still shows a moderate negative correlation with danceability (instrumental tracks tend to be somewhat less danceable) and a moderate positive correlation with acousticness (instrumental songs are often more acoustic, e.g., orchestral or ambient).

Partial Correlations

Thanks to the Partial Correlation we can investigate the **relationship between two variables without the influence of any other relationship between them and other attributes present in the dataset**. These relations can be clearly represented by the following plots.

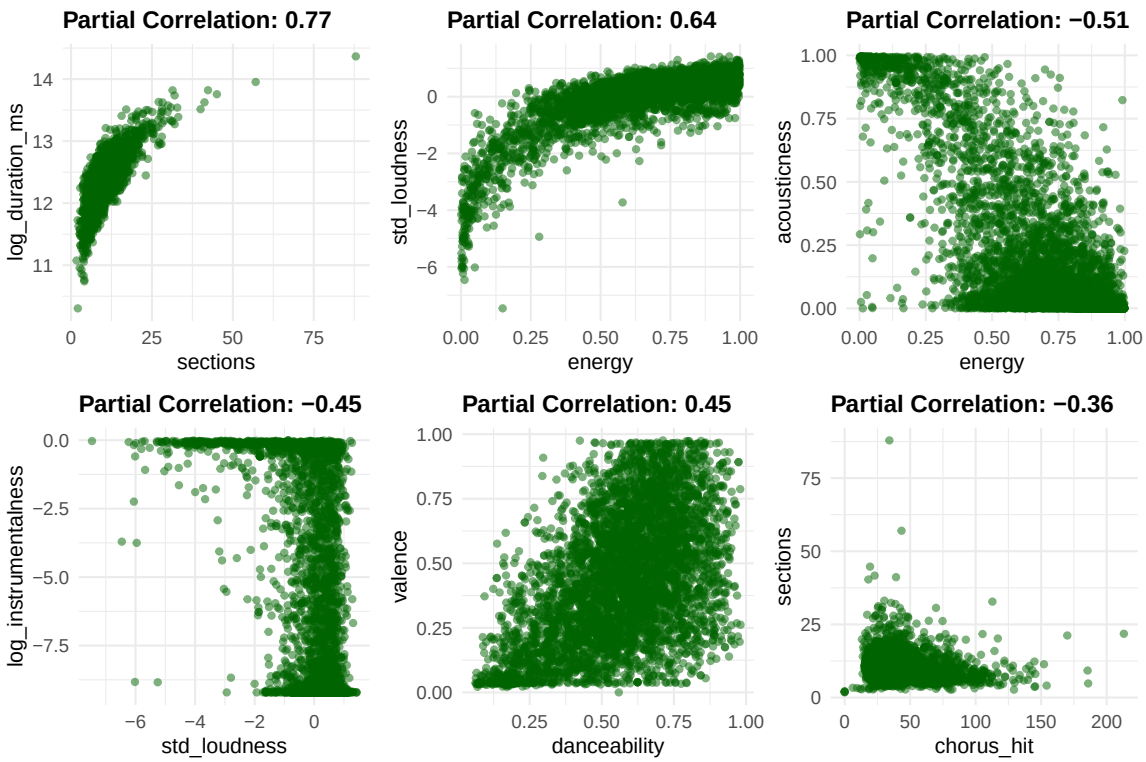
```r
partial_corr <- correlation(train %>% select(where(is.numeric)), partial = TRUE)
# absolute values > 0.3
cat("Partial Correlations with absolute values > 0.3:\n")
```

```
## Partial Correlations with absolute values > 0.3:
```

```r
top_partial_corr <- partial_corr %>%
  filter(abs(r) > 0.3) %>%
  arrange(desc(abs(r)))
print(top_partial_corr)
```

```
## # Correlation Matrix (pearson-method)
##
## Parameter1   |           Parameter2 |     r |         95% CI | t(4796) |         p
## ---------------------------------------------------------------------------------
## sections     |      log_duration_ms |  0.77 | [ 0.76,  0.78] |   84.20 | < .001***
## energy       |         std_loudness |  0.64 | [ 0.62,  0.65] |   57.25 | < .001***
## energy       |         acousticness | -0.51 | [-0.53, -0.49] |  -41.25 | < .001***
## std_loudness | log_instrumentalness | -0.45 | [-0.48, -0.43] |  -35.26 | < .001***
## danceability |              valence |  0.45 | [ 0.42,  0.47] |   34.58 | < .001***
## chorus_hit   |             sections | -0.36 | [-0.39, -0.34] |  -26.91 | < .001***
## danceability |               energy | -0.34 | [-0.37, -0.32] |  -25.24 | < .001***
## chorus_hit   |      log_duration_ms |  0.32 | [ 0.30,  0.35] |   23.68 | < .001***
## energy       |              valence |  0.31 | [ 0.28,  0.33] |   22.28 | < .001***
##
## p-value adjustment method: Holm (1979)
## Observations: 4798
```

```r
# Selecting the top 6 most significant partial correlations
top_pairs <- top_partial_corr %>% slice(1:6)
# Generating the scatter plots with jitter for better visualization
plots <- lapply(1:nrow(top_pairs), function(i) {
  ggplot(train, aes(x = .data[[top_pairs$Parameter1[i]]], y = .data[[top_pairs$Parameter2[i]]])) +
    geom_jitter(color = "darkgreen", alpha = 0.5, size = 0.8) +
    labs(
      x = top_pairs$Parameter1[i],
      y = top_pairs$Parameter2[i],
      title = paste("Partial Correlation:", round(top_pairs$r[i], 2))
    ) +
    theme_minimal(base_size = 8) +  # Dimensione caratteri ridotta
    theme(
      plot.title = element_text(size = 9, face = "bold"),
      axis.title = element_text(size = 8),
      axis.text = element_text(size = 7)
    )
})
# Arranging plots in a 2x3 grid
grid.arrange(grobs = plots, ncol = 3, nrow = 2)
```



Variables	Partial Corr.	Interpretation
Sections & log_duration_ms	**+0.77**	Longer songs tend to have more sections.
Energy & std_loudness	**+0.64**	Higher energy tracks are naturally louder.
Acousticness & Energy	**−0.51**	Acoustic tracks typically have lower energy.
log_instrumentalness & std_loudness	**−0.45**	Instrumental tracks tend to be quieter.
Valence & Danceability	**+0.45**	Upbeat tracks are generally more danceable.
Chorus_hit & Sections	**−0.36**	More sections may lead to earlier chorus placement.

Implications for Modeling: High correlations, such as energy & loudness or sections & duration, can lead to *multicollinearity*, affecting model stability and interpretability. This issue is particularly relevant in logistic regression (inflated variance) and KNN (distorted distance metrics).

To *mitigate* this, we will: - Use Regularization (Ridge/Lasso) to shrink redundant coefficients.

- Apply feature Selection (e.g., stepwise regression, VIF filtering) to remove highly correlated variables.

While collinearity isn't inherently problematic, it requires careful handling, especially in linear and distance-based models. In the following sections, we outline our approach to managing these dependencies for more robust and interpretable predictions.

Logistic Regression Models Definition

Logistic Regression is applied to classify songs as Hit or Flop, using both the original training datasets. A stepwise selection approach is explored to optimize feature selection.

To minimize false positives (flops misclassified as hits), *multiple classification thresholds are evaluated* to optimize resource allocation and maintain a balance between false positives and false negatives.

The following sections outline the modeling approach, comparing standard vs. stepwise regression, while selecting optimal thresholds to reduce misclassification impact.

```r
library(ROSE)
```

```
## Warning: il pacchetto 'ROSE' è stato creato con R versione 4.4.2
```

```
## Loaded ROSE 0.0-4
```

```r
library(caret)
set.seed(1)
train_balanced <- ovun.sample(target ~ ., data = train, method = "both",
                              p = 0.5,
                              N = nrow(train),
                              seed = 1)$data
#Classification Tresholds
threshold4 <- 0.4
threshold5 <- 0.5
threshold6 <- 0.6
```

```r
cat("\n Class proportion:\n")
```

```
##
##  Class proportion:
```

```r
print(prop.table(table(train_balanced$target)))
```

```
##
##         0         1
## 0.5147978 0.4852022
```

Simple Logistic Model

Handling Multicollinearity in Logistic Regression: A **full logistic regression model (glm_full)** was initially built using all available predictors to estimate a song's probability of being a Hit. However, multicollinearity—strong correlations among predictors—can inflate coefficient variance, affecting model stability.

Variance Inflation Factor (VIF) Analysis: To detect collinearity, the Variance Inflation Factor (VIF) was computed. A VIF exceeding $1/(1-R^2)$ 1.699 indicates that a predictor is more correlated with other features than with the response. The analysis identified:

- *sections (VIF = 2.54)*
- *std_loudness (VIF = 2.53)* Since these features showed excessive redundancy, they were iteratively removed to improve model reliability. The final model (glm_final) retains log_duration_ms (VIF = 2.55), balancing the reduction of collinearity while preserving valuable predictive information.

```r
library(car)
```

```r
glm_full <- glm(target ~ ., data = train_balanced, family = "binomial")
# R-Squared and VIF
s <- summary(glm_full)
r2 <- 1 - (s$deviance / s$null.deviance)
vif_threshold <- 1 / (1 - r2)
cat("\n R-squared for complete model:", round(r2, 3), "\n")
```

```
##
##  R-squared for complete model: 0.411
```

```r
cat(" VIF:", round(vif_threshold, 3), "\n")
```

```
##  VIF: 1.699
```

```r
vif_values <- vif(glm_full)
cat("\n VIF value for each variable:\n")
```

```
##
##  VIF value for each variable:
```

```r
print(round(vif_values, 2))
```

```
##         danceability               energy                  key
##                 1.45                 3.59                 1.02
##                 mode          speechiness         acousticness
##                 1.05                 1.13                 1.77
##             liveness              valence       time_signature
##                 1.10                 1.53                 1.06
##           chorus_hit             sections         std_loudness
##                 1.27                 2.54                 2.53
##            std_tempo      log_duration_ms log_instrumentalness
##                 1.11                 2.55                 1.15
```

```r
glm_final <- glm(target ~ . - sections - std_loudness,
                 data = train_balanced,
                 family = "binomial")
cat("\n *Final Model*:\n")
```

```
##
##  *Final Model*:
```

```r
print(summary(glm_final))
```

```
##
## Call:
## glm(formula = target ~ . - sections - std_loudness, family = "binomial",
##     data = train_balanced)
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)          -3.479286   2.097265  -1.659 0.097123 .
## danceability          4.129155   0.302155  13.666  < 2e-16 ***
## energy               -1.310521   0.283396  -4.624 3.76e-06 ***
## key                   0.002038   0.010694   0.191 0.848837
## mode                  0.274991   0.082758   3.323 0.000891 ***
## speechiness          -0.373366   0.400613  -0.932 0.351343
## acousticness         -1.673914   0.216710  -7.724 1.13e-14 ***
## liveness             -0.708855   0.233677  -3.033 0.002417 **
## valence              -1.349207   0.205749  -6.558 5.47e-11 ***
## time_signature        0.516537   0.124739   4.141 3.46e-05 ***
## chorus_hit           -0.003443   0.002083  -1.653 0.098257 .
## std_tempo             0.178097   0.040938   4.350 1.36e-05 ***
## log_duration_ms      -0.201449   0.159898  -1.260 0.207721
## log_instrumentalness -0.487181   0.017664 -27.581  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 6647.2  on 4797  degrees of freedom
## Residual deviance: 4113.9  on 4784  degrees of freedom
## AIC: 4141.9
##
## Number of Fisher Scoring iterations: 6
```

- **Significant Attributes**
  - **danceability** (`+4.13`), **acousticness** (`-1.67`), **valence** (`-1.35`), **time_signature** (`+0.52`), **std_tempo** (`+0.18`), and **log_instrumentalness** (`-0.49`) stand out with high significance.
  - **mode** (`+0.27`), **energy** (`-1.31`), and **liveness** (`-0.71`) also show strong effects.

- Variables like `key`, `speechiness`, and `log_duration_ms` do not appear as significant at the 5% level, but we retain them to avoid omitting potentially relevant info or to maintain stable estimates.

Predictions on data:

```
predictions <- predict(glm_final, test, type = "response")

thresholds <- c(0.4, 0.5, 0.6)
results <- list()

for (threshold in thresholds) {
  pred_classes <- ifelse(predictions > threshold, 1, 0)
  conf_matrix <- table(test$target, pred_classes)
  misclassification_rate <- mean(pred_classes != as.numeric(as.character(test$target)))
  results[[paste0("Threshold_", threshold)]] <- list(
    ConfusionMatrix = conf_matrix,
    MisclassificationRate = misclassification_rate
  )

  cat("\n Resaults for treshold =", threshold, ":\n")
  print(conf_matrix)
  cat(" Misclassification Rate:", round(misclassification_rate, 4), "\n")
}

#  Risultati completi
cat("\n Final resaults for each treshold:\n")
print(results)
```

Confusion Matrices for Simple Logistic Regression

Threshold	**TN** (Flop→Flop)	**FP** (Flop→Hit)	**FN** (Hit→Flop)	**TP** (Hit→Hit)	**Total**	**Misclass. Rate**
0.4	534	264	63	739	1600	**20.44%**
0.5	572	226	87	715	1600	**19.56%**
0.6	616	182	150	652	1600	**20.75%**

where: - **TN (Flop→Flop):** Actual flop tracks correctly identified as flop.
- **FP (Flop→Hit):** Actual flop tracks mislabeled as hit.
- **FN (Hit→Flop):** Actual hit tracks missed (predicted as flop).
- **TP (Hit→Hit):** Actual hit tracks correctly labeled as hit.
- **Misclass. Rate:** $\frac{\text{FP+FN}}{\text{Total}}$

Observations:

- At **threshold = 0.4**, we see the fewest false negatives (63) but more false positives (264), leading to a 20.44% overall error.
- At **threshold = 0.5**, false positives drop (226), but false negatives rise to 87, producing the lowest misclassification rate (19.56%).
- At **threshold = 0.6**, the model becomes more conservative about calling a track a hit, further reducing false positives (182) but increasing false negatives (150), resulting in a 20.75% overall error.

Focusing on the confusion matrices, we see that **increasing** the threshold for classifying a track as a *hit* **reduces** false positives (tracks incorrectly labeled as hit) but **increases** false negatives (actual hits missed). Conversely, if we **lower** the threshold, we capture more potential hits but risk recommending flop tracks—raising the false positive count. The choice of an optimal threshold thus depends on which error type (missing a real hit vs. wrongly promoting a flop) is more critical for our application.

Logistic Model with Stepwise Selection

After constructing a full logistic regression model, we introduced a stepwise selection process using the stepAIC() function (from the MASS package) to minimize the Akaike Information Criterion (AIC). This method iteratively adds or removes predictors based on their contribution to the model's goodness of fit, penalizing an excessive number of variables:

We started with the full model (all variables included) as described earlier. Stepwise Approach (direction = "both"). This combines forward and backward strategies, adding highly contributive predictors and removing those that no longer improve the model's AIC.

```r
library(MASS)
library(caret)
```

```r
glm_full <- glm(target ~ ., data = train_balanced, family = "binomial")
# Minimize AIC
glm_stepwise <- stepAIC(glm_full, direction = "both", trace = FALSE)
cat("\n Stepwise Model Summary:\n")
```

```
##
##  Stepwise Model Summary:
```

```r
summary(glm_stepwise)
```

```
##
## Call:
## glm(formula = target ~ danceability + energy + mode + acousticness +
##     liveness + valence + time_signature + chorus_hit + sections +
##     std_loudness + std_tempo + log_duration_ms + log_instrumentalness,
##     family = "binomial", data = train_balanced)
##
## Coefficients:
##                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)          -10.847440   3.079496  -3.522 0.000428 ***
## danceability           3.664014   0.310450  11.802  < 2e-16 ***
## energy                -4.756698   0.393100 -12.100  < 2e-16 ***
## mode                   0.252489   0.084093   3.003 0.002678 **
## acousticness          -1.578097   0.226078  -6.980 2.95e-12 ***
## liveness              -0.441780   0.239821  -1.842 0.065457 .
## valence               -1.060325   0.213161  -4.974 6.55e-07 ***
## time_signature         0.655055   0.133654   4.901 9.53e-07 ***
## chorus_hit            -0.006911   0.002386  -2.896 0.003774 **
## sections              -0.083259   0.021870  -3.807 0.000141 ***
## std_loudness           1.632207   0.126809  12.871  < 2e-16 ***
## std_tempo              0.177011   0.041908   4.224 2.40e-05 ***
## log_duration_ms        0.649336   0.256474   2.532 0.011348 *
## log_instrumentalness  -0.410961   0.018392 -22.345  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 6647.2  on 4797  degrees of freedom
## Residual deviance: 3913.3  on 4784  degrees of freedom
## AIC: 3941.3
##
## Number of Fisher Scoring iterations: 6
```

Model Comparison and Interpretation **AIC Improvement**: *The stepwise model's lower AIC (~3941.3) suggests it fits the data better when balancing complexity and performance.*

Similar Core Predictors: Many highly influential variables—like danceability, acousticness, and log_instrumentalness—remain from prior models, underscoring their consistent importance for predicting hit vs. flop.

Reintroduced Variables: 1. **sections** reappears with a negative coefficient, suggesting that, once the penalty for too many variables is balanced, the model benefits from including it. 2. **std_loudness** emerges as a strongly positive term, indicating louder tracks after standardization are more likely to become hits. The stepwise approach confirms many of our earlier findings but refines the set of features to optimize AIC.

```r
predictions_stepwise <- predict(glm_stepwise, test, type = "response")
thresholds <- c(0.4, 0.5, 0.6)
results_stepwise <- list()
for (threshold in thresholds) {
  # Conversion of probabilities into binary classes (hit/flop)
  pred_classes <- ifelse(predictions_stepwise > threshold, 1, 0)
  conf_matrix <- table(test$target, pred_classes)
    misclassification_rate <- mean(pred_classes != as.numeric(as.character(test$target)))
```

```r
  results_stepwise[[paste0("Threshold_", threshold)]] <- list(
    ConfusionMatrix = conf_matrix,
    MisclassificationRate = misclassification_rate
  )
    cat("\n  Results for threshold =", threshold, ":\n")
  print(conf_matrix)
  cat("  Misclassification rate:", round(misclassification_rate, 4), "\n")
}
cat("\n  Final results of the Stepwise model for each threshold:\n")
print(results_stepwise)
```

Confusion Matrices (*Stepwise Logistic Regression*):

Threshold	TN	FP	FN	TP	Total	Misclassification
0.4	554	244	58	744	1600	18.88%
0.5	592	206	86	716	1600	18.25%
0.6	643	155	140	662	1600	18.44%

Observations:

1. **Lowest Overall Error at Threshold 0.5**
   The misclassification rate is approximately **18.25%** —slightly better than the other two thresholds.
2. **Trade-off: False Positives vs. False Negatives**
   - At **threshold = 0.4**, the model is more liberal in predicting hits, which reduces *missed hits* (FN = 58) but boosts *false positives* (FP = 244).
   - At **threshold = 0.6**, fewer flop tracks are mistaken for hits (FP = 155), but the model overlooks more actual hits (FN = 140).

Stepwise vs. Simple Model
Compared to the simple logistic model, the **stepwise** approach yields slightly **lower misclassification rates overall, indicating that the subset of predictors chosen through AIC minimization provides a marginal improvement in balancing hits vs. flops** . Ultimately, your choice of threshold depends on whether **missing a real hit** (false negative) is more costly than **promoting a flop** (false positive).

Weighted GLM

Instead of modifying the dataset composition, *class weights are assigned to address class imbalance. If the Flop class is underrepresented or its misclassification carries higher consequences, increasing its weight compels the model to prioritize its correct classification, reducing bias toward the majority class.

In the following section, weighted logistic regression is implemented, applying a higher penalty for misclassifying flops. This ensures that both classes contribute equally to parameter estimation while preserving the full dataset, making it particularly effective for adjusting the decision boundary based on class importance.

```
## Weight assigned to the less represented class: 1
```

Weight assigned to thelessrepresentedclass:1

Create the new model and make some predictions:

```r
glm_weighted <- glm(target ~ ., data = train, family = "binomial", weights = weights)
```

```
## Warning in eval(family$initialize): #successi non interi in un binomial glm!
```

```r
cat("\n Summary of the weighted model:\n")
print(summary(glm_weighted))

pred_weighted <- predict(glm_weighted, test, type = "response")
thresholds <- c(0.4, 0.5, 0.6)
results_weighted <- list()

#Model evaluation for each threshold
for (threshold in thresholds) {
  pred_classes <- ifelse(pred_weighted > threshold, 1, 0)
  conf_matrix <- table(test$target, pred_classes)
  misclassification_rate <- mean(pred_classes != as.numeric(as.character(test$target)))

  results_weighted[[paste0("Threshold_", threshold)]] <- list(
    ConfusionMatrix = conf_matrix,
    MisclassificationRate = misclassification_rate
  )
  cat("\n Results for threshold =", threshold, ":\n")
  print(conf_matrix)
  cat(" Misclassification rate:", round(misclassification_rate, 4), "\n")
}
cat("\n Final results of the weighted model for each threshold:\n")
print(results_weighted)
```

Overall, while the fundamental story—vocal, less acoustic, and danceable tracks typically fare better—remains consistent, **new or reintroduced predictors (like sections, std_loudness, or time_signature) in the weighted model reflect how adjusting the algorithm's priorities (weights) or penalizing unnecessary complexity (stepwise) can reshape which variables ultimately matter in the final classifier**.

We look now at the *confusion matrices* in order to understand if this new model could be a good alternative to the two previous proposed:

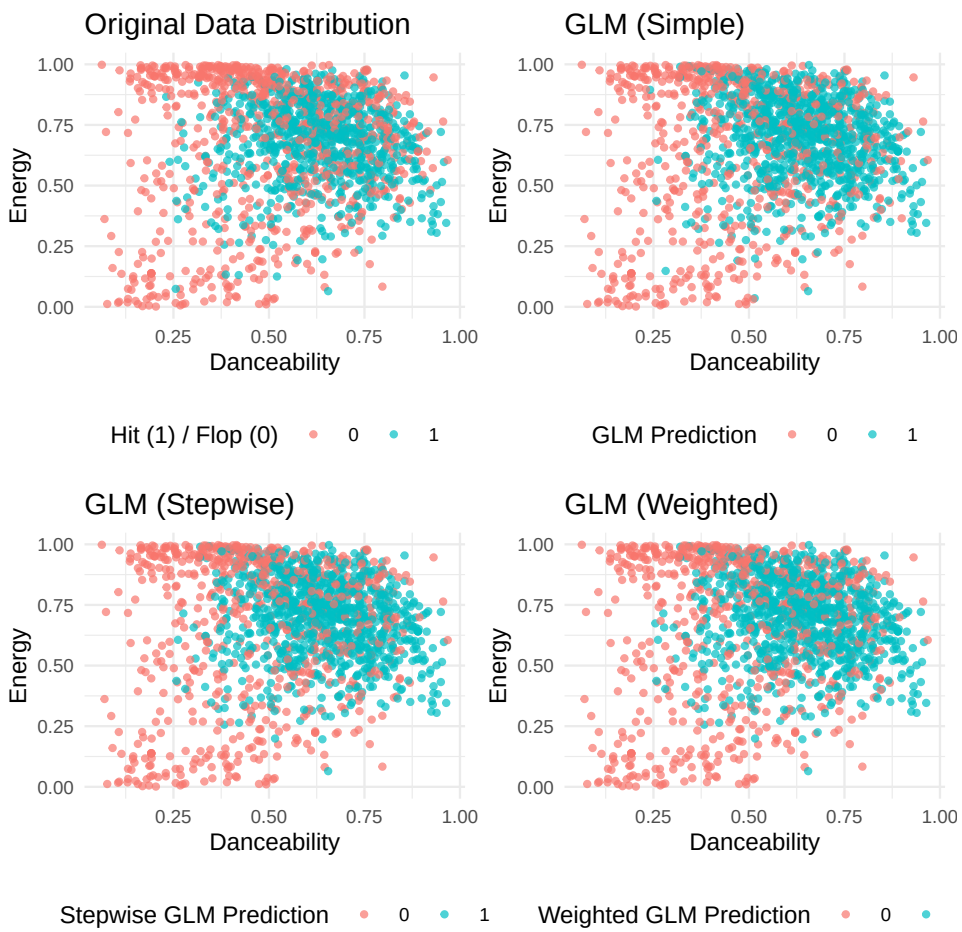
Threshold	TN	FP	FN	TP	Total	Misclass. Rate
0.4	543	255	56	746	1600	19.44%
0.5	583	215	82	720	1600	18.56%
0.6	635	163	127	675	1600	18.12%

By assigning a higher weight to the **Flop** class, the model pays extra attention to flop misclassifications during training, thereby subtly shifting decision boundaries. **In this case, threshold = 0.6 yields the lowest total error (18.12%), indicating that a more conservative approach to labeling hits can be effective under weighted conditions.** However, whether this is preferable depends on the cost trade-off between missing potential hits (FN) and incorrectly promoting flops (FP).

The classification performance of the models is assessed by evaluating how well they replicate the original graphical classification of the points, as illustrated in the following plots.

```r
# Original Data Distribution
original_plot <- ggplot(test, aes(x = danceability, y = energy, color = as.factor(target))) +
  geom_point(alpha = 0.7, size = 0.8) +
  labs(x = "Danceability", y = "Energy", color = "Hit (1) / Flop (0)",
       title = "Original Data Distribution") +
  theme_minimal(base_size = 9) +
  theme(legend.position = "bottom")

# Simple GLM Predictions
plot_glm <- ggplot(test, aes(x = danceability, y = energy, color = as.factor(ifelse(predictions > 0.5, 1, 0)))) +
  geom_point(alpha = 0.7, size = 0.8) +
  labs(x = "Danceability", y = "Energy", color = "GLM Prediction",
       title = "GLM (Simple)") +
  theme_minimal(base_size = 9) +
  theme(legend.position = "bottom")

# GLM with Stepwise Selection
plot_stepwise <- ggplot(test, aes(x = danceability, y = energy, color = as.factor(ifelse(predictions_stepwise > 0.5
  geom_point(alpha = 0.7, size = 0.8) +
```

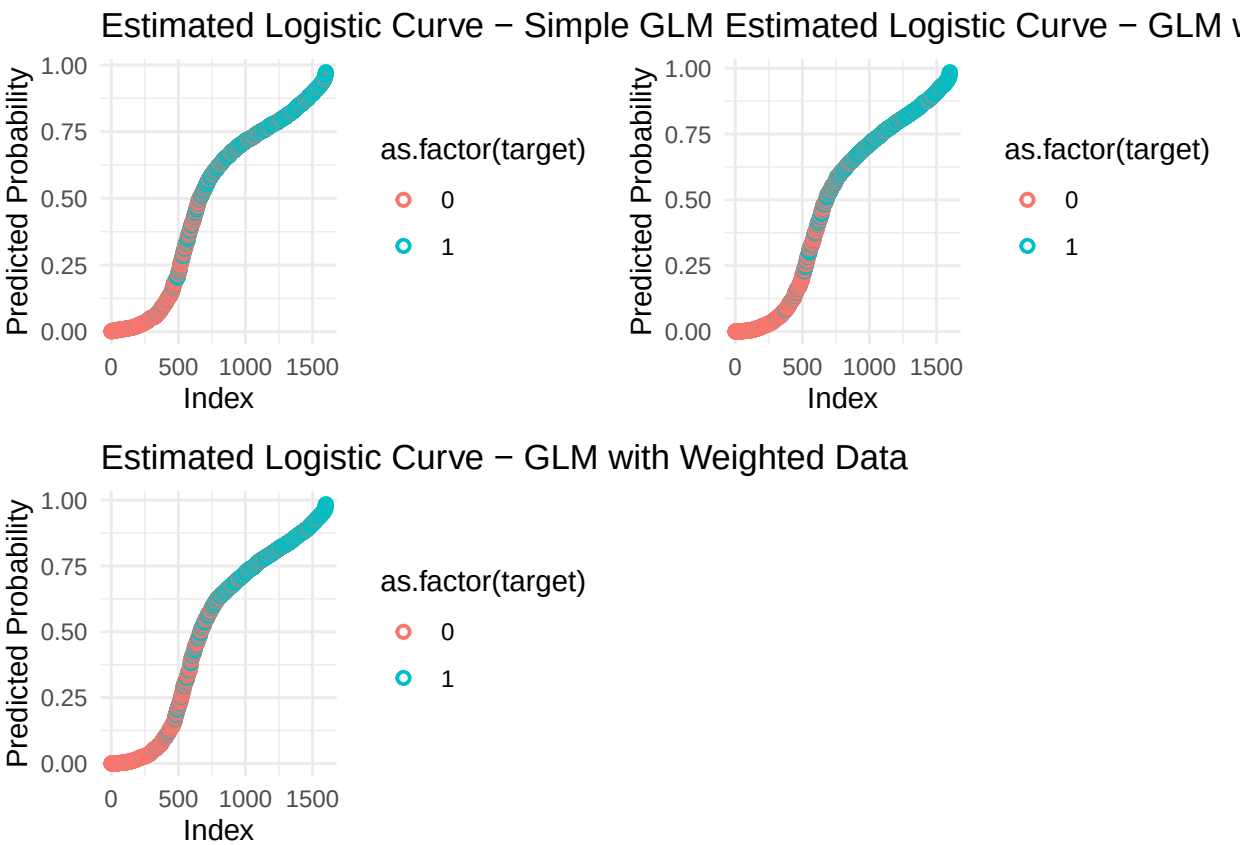
```r
  labs(x = "Danceability", y = "Energy", color = "Stepwise GLM Prediction",
       title = "GLM (Stepwise)") +
  theme_minimal(base_size = 9) +
  theme(legend.position = "bottom")
# GLM with Weights
plot_weighted <- ggplot(test, aes(x = danceability, y = energy, color = as.factor(ifelse(pred_weighted > 0.5, 1, 0
  geom_point(alpha = 0.7, size = 0.8) +
  labs(x = "Danceability", y = "Energy", color = "Weighted GLM Prediction",
       title = "GLM (Weighted)") +
  theme_minimal(base_size = 9) +
  theme(legend.position = "bottom")
# Arrange Plots in a 2x2 Grid
grid.arrange(original_plot, plot_glm, plot_stepwise, plot_weighted, ncol = 2, nrow = 2)
```



The major differences often appear when you contrast the original distribution (which is ground truth) with the predicted classifications from each model. If the stepwise and weighted variations do not drastically modify the logistic decision boundary, the scatter plots may look quite similar. The weighting might reduce false positives (flops incorrectly labeled as hits) but could slightly increase false negatives (missed hits). Stepwise might yield a marginal improvement if certain redundant features are dropped, but the effect can be subtle. In any case, the difference between all the GLM models is not so relevant.

```r
#Compare the results obtained with the four different models, plotting
#now an estimation of the logistic curve using the predictions.
plot_logistic_curve <- function(predictions, title) {
  predicted_data <- data.frame(prob_of_hit = predictions, target = test$target)
  predicted_data <- predicted_data[order(predicted_data$prob_of_hit, decreasing = FALSE),]
```

```r
  predicted_data$rank <- 1:nrow(predicted_data)
  ggplot(data = predicted_data, aes(x = rank, y = prob_of_hit)) +
    geom_point(aes(color = as.factor(target)), alpha = 1, shape = 1, stroke = 1) +
    xlab("Index") +
    ylab("Predicted Probability") +
    ggtitle(title) +
    theme_minimal()
}
plot_glm_curve <- plot_logistic_curve(predictions, "Estimated Logistic Curve - Simple GLM")
plot_stepwise_curve <- plot_logistic_curve(predictions_stepwise, "Estimated Logistic Curve - GLM with Stepwise")
plot_weighted_curve <- plot_logistic_curve(pred_weighted, "Estimated Logistic Curve - GLM with Weighted Data")
grid.arrange(plot_glm_curve, plot_stepwise_curve, plot_weighted_curve, nrow = 2)
```



The logistic probability curves provide insight into how different models classify tracks as hits or flops. Each song in the test set is positioned along the x-axis, sorted by its predicted probability of being a hit, while the y-axis represents that probability.

Across all models—simple, stepwise, and weighted logistic regression—songs with high predicted probabilities are almost always classified as hits, and those with low probabilities are labeled as flops. However, the uncertainty zone (0.4–0.6 probability range) reveals how each model handles borderline cases, where classification is less clear.

Comparing Model Performance: The weighted (balanced) model more accurately reflects the actual distribution of hits and flops, as illustrated by the colored points (red = flop, teal = hit) along the logistic curve. By analyzing the number of teal (hit) points above a given threshold (e.g., 0.5), it becomes evident that the balanced training approach prevents the model from being overly aggressive or conservative in classifying hits.

The confusion matrices further support this observation: the balanced model reduces false negatives (fewer missed hits) while, depending on weighting, controlling excessive false positives. In contrast, standard and stepwise GLM models may misalign the hit-to-flop ratio, particularly when trained on an unbalanced dataset.

Discriminant Analysis

Beyond generalized linear models, another valuable approach to classification is **Discriminant Analysis**, including both **Linear Discriminant Analysis (LDA)** and **Quadratic Discriminant Analysis (QDA)**. These methods draw from **Bayes' theorem** to assign an observation to the class that maximizes its posterior probability, given the features.

In our case, we wish to distinguish between **Hit** and **Flop** songs based on audio features. Discriminant Analysis offers a different perspective compared to logistic regression, potentially capturing additional structure in the data—particularly if the assumptions underlying LDA or QDA hold.

Normality Requirement

A primary assumption for **LDA** is that the **predictor variables (covariates) are normally distributed** within each class (Hit vs. Flop). **QDA** similarly relies on normality but allows each class to have a distinct covariance matrix.

Shapiro-Wilk Test

To check whether our numeric features meet this normality requirement within each class, we use the **Shapiro-Wilk test**. For each feature, we split the data into Hit vs. Flop subsets and test each subset's distribution against the null hypothesis of normality:

- **Null Hypothesis (H)**: The sample comes from a normal distribution.
- **Alternative Hypothesis (H)**: The sample does *not* come from a normal distribution.

If the **p-value** is 0.05, we reject H and conclude that the feature in that class does not appear normally distributed at the 95% confidence level. If many key features strongly violate normality, we may have to reconsider using LDA/QDA or explore transformations that bring the data closer to normality.

In the following sections, we detail the **Shapiro-Wilk tests** for each numeric feature under the Hit and Flop classes, discuss whether the data meets LDA/QDA assumptions, and then proceed with implementing LDA/QDA on our dataset if conditions appear sufficiently met (or at least not severely violated).

```r
numeric_vars <- train_balanced[, sapply(train_balanced, is.numeric)]
numeric_vars <- numeric_vars[, !(colnames(numeric_vars) %in% "target")]

shapiro_results <- lapply(names(numeric_vars), function(var) {
  hit_data <- train_balanced[[var]][train_balanced$target == 1]
  flop_data <- train_balanced[[var]][train_balanced$target == 0]

  hit_test <- shapiro.test(hit_data)
  flop_test <- shapiro.test(flop_data)

  list(
    Variable = var,
    Hit_W = hit_test$statistic,
    Hit_p_value = hit_test$p.value,
    Flop_W = flop_test$statistic,
    Flop_p_value = flop_test$p.value
  )
})
shapiro_df <- do.call(rbind, lapply(shapiro_results, as.data.frame))
rownames(shapiro_df) <- NULL
print(shapiro_df)
```

```
##                Variable     Hit_W  Hit_p_value    Flop_W Flop_p_value
## 1          danceability 0.9961480 1.124235e-05 0.9826730 8.652660e-17
## 2                energy 0.9691702 5.972196e-22 0.8964742 6.238524e-38
## 3                   key 0.9168790 5.587250e-34 0.9281107 6.961947e-33
## 4                  mode 0.5957905 6.588052e-59 0.6159724 3.421585e-59
## 5           speechiness 0.7478158 9.880376e-51 0.6200680 5.344745e-59
## 6          acousticness 0.7503977 1.465897e-50 0.7450835 5.065907e-52
## 7              liveness 0.7500411 1.387876e-50 0.7280712 4.035299e-53
## 8               valence 0.9846589 3.822080e-15 0.9440712 1.254254e-29
## 9        time_signature 0.2545592 1.874858e-70 0.4486720 6.458249e-66
```

```
## 10           chorus_hit 0.8568189 9.414067e-42 0.8538986 4.794376e-43
## 11             sections 0.9645279 1.750559e-23 0.7942549 1.838732e-48
## 12         std_loudness 0.9325810 3.291447e-31 0.8128471 6.255771e-47
## 13            std_tempo 0.9829326 4.273994e-16 0.9816094 2.387404e-17
## 14      log_duration_ms 0.9666008 8.092038e-23 0.9843966 7.878998e-16
## 15 log_instrumentalness 0.4589586 2.824259e-64 0.7976321 3.420002e-48
```

To evaluate whether the numeric features follow a normal distribution within each class (Hit vs. Flop), Shapiro-Wilk tests were conducted on each variable. The analysis revealed that:

- **Most variables fail the normality test**, with p-values well below 0.05, indicating significant deviations from a normal distribution.
- W statistics for many features are far from 1.0, confirming pronounced skewness or **non-normal patterns**, particularly in variables such as danceability, energy, mode, and time_signature.
- Log transformations and linear scaling were applied to correct skewness; however, Shapiro-Wilk tests still rejected normality, indicating that key features remain non-normally distributed even after adjustments.

The lack of normality may affect predictive performance, particularly for Linear Discriminant Analysis (LDA), which assumes normally distributed features and relies on linear decision boundaries. In contrast, Quadratic Discriminant Analysis (QDA), which accommodates class-specific covariance structures, may offer greater flexibility in handling non-normal data.

Additionally, tree-based and nonparametric models (e.g., KNN, Random Forests) may be more robust in this context, as they do not impose normality assumptions on the predictors.

Despite these *limitations*, we *proceed with LDA/QDA*, acknowledging that their theoretical optimality may not be fully realized. The results will be analyzed with caution, and comparisons with alternative models will help determine their practical viability.

Despite these limitations, LDA and QDA are applied, with the understanding that their theoretical assumptions may not be fully met. The results will be carefully interpreted, and comparisons with alternative models will provide insight into their practical effectiveness in classifying songs as hits or flops.

Linear Discriminant Analysis

With Linear Discriminant Analysis, we aim to find a linear combination of our features that best separates the two classes—Hit (1) and Flop (0)—and use that combination as a linear classifier. In applying LDA, we must remember that it ideally operates on independent features. Therefore, our initial LDA model will exclude variables known to be collinear from prior analyses (e.g., identified via VIF or correlation checks). The same principle will apply to Quadratic Discriminant Analysis (QDA), discussed later.

Simple LDA We define now a Simple LDA model, as we said before, starting from the knowledge about the best subset ofvariables to use deriving from the analysis conduced with the use ofVIF.

```r
library(MASS)
library(ggplot2)
library(dplyr)
```

```r
lda_model <- lda(target ~ . - sections - std_loudness, data = train_balanced)
cat("\n LDA Summary:\n")
```

```
##
##  LDA Summary:
```

```r
print(lda_model)
```

```
## Call:
## lda(target ~ . - sections - std_loudness, data = train_balanced)
##
## Prior probabilities of groups:
##         0         1
## 0.5147978 0.4852022
##
## Group means:
```

```
##   danceability    energy      key      mode speechiness acoustic ness  liveness
## 0    0.4830267 0.6416010 5.175709 0.6182186  0.08427239    0.2844512 0.2047404
## 1    0.6453909 0.6881589 5.273196 0.6649485  0.11004308    0.1528822 0.1874717
##     valence time_signature chorus_hit   std_tempo log_duration_ms
## 0 0.3891232       3.876113   42.67464 -0.07755308        12.36368
## 1 0.4992860       3.983677   39.02287  0.07621610        12.28066
##   log_instrumentalness
## 0            -4.173572
## 1            -8.629906
##
## Coefficients of linear discriminants:
##                                LD1
## danceability          1.922375372
## energy               -1.118628266
## key                   0.002240978
## mode                  0.127059874
## speechiness          -0.131729425
## acousticness         -0.946821184
## liveness             -0.487460766
## valence              -0.649350362
## time_signature        0.181585313
## chorus_hit           -0.003091152
## std_tempo             0.081743364
## log_duration_ms      -0.147265862
## log_instrumentalness -0.291981763
```

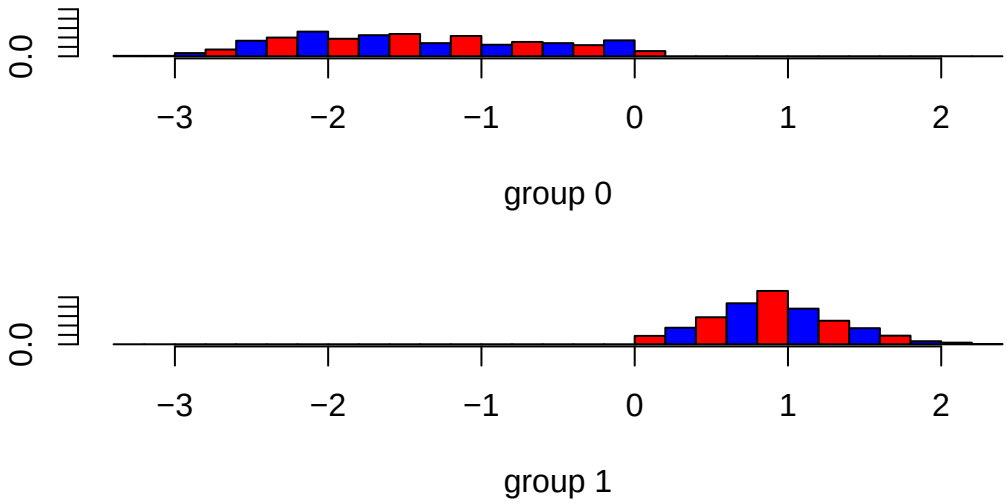
```r
pred_lda <- predict(lda_model, test)
posterior_lda <- pred_lda$posterior

thresholds <- c(0.4, 0.5, 0.6, 0.2, 0.3,0.7)
predictions_lda <- list()
conf_matrices <- list()
misclassification_rates <- c()
```

The LDA summary highlights **danceability, acousticness, log_instrumentalness, and time_signature as the most influential features in distinguishing Hit from Flop tracks**. Higher danceability and time signature push a song toward Hit, while acousticness and instrumentalness increase the likelihood of a Flop.

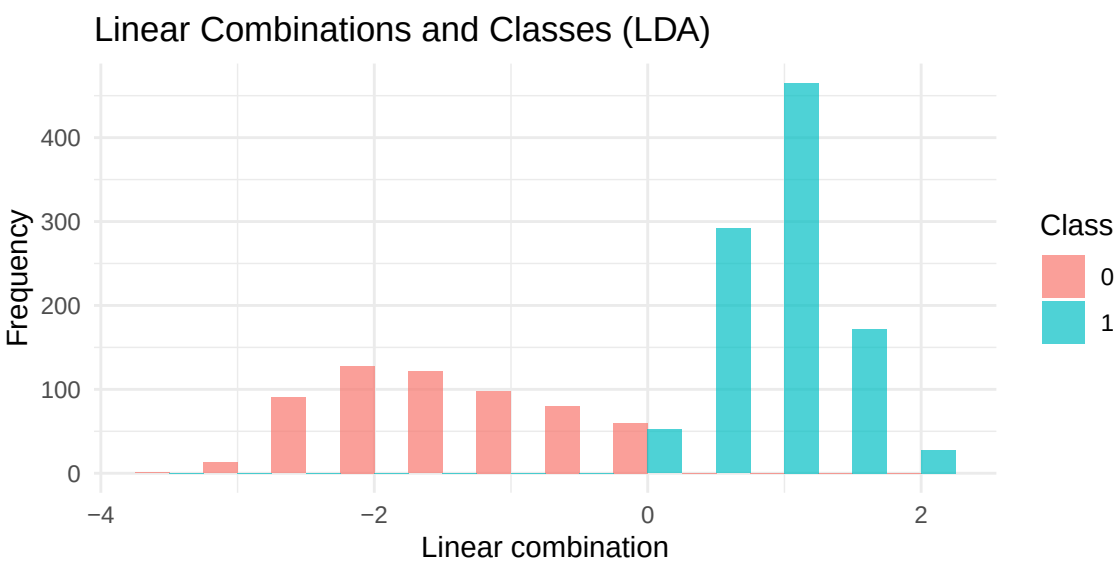
Rather than a single defining factor, it is the combination of these features that shapes LDA's classification boundary. To assess its effectiveness, we analyze the **confusion matrix** across different thresholds and examine the distribution of discriminant scores. This will help determine whether LDA provides a clear separation between classes or if adjustments are needed for better accuracy.

```r
for (threshold in thresholds) {
  pred_classes <- as.factor(ifelse(posterior_lda[, 2] > threshold, 1, 0))
  conf_matrix <- table(test$target, pred_classes)
  misclassification_rate <- mean(pred_classes != test$target)

  predictions_lda[[paste0("Threshold_", threshold)]] <- pred_classes
  conf_matrices[[paste0("Threshold_", threshold)]] <- conf_matrix
  misclassification_rates <- c(misclassification_rates, misclassification_rate)

  cat("\n Results for threshold =", threshold, ":\n")
  print(conf_matrix)
  cat("Error rate:", round(misclassification_rate, 4), "\n")
}
```

```r
par(mar=c(4, 4, 2, 1))  # Reduce margins (bottom, left, top, right)
ldahist(pred_lda$x[, 1], g = pred_lda$class, col = c("blue", "red"))
```




```
combination_plot <- ggplot(data.frame(x = pred_lda$x[, 1], class = pred_lda$class),
                           aes(x = x, fill = as.factor(class))) +
  geom_histogram(binwidth = 0.5, position = "dodge", alpha = 0.7) +
  labs(x = "Linear combination", y = "Frequency", fill = "Class",
       title = "Linear Combinations and Classes (LDA)") +
  theme_minimal()
print(combination_plot)
```




```
lda_results <- data.frame(
  Threshold = thresholds,
  Misclassification_Rate = misclassification_rates
)
cat("\n Results:\n")
print(lda_results)
```

LDA Model Results by Threshold

Model	Overall Error Rate	False Negative Rate	Precision (Hit)
LDA - 0.4	**21.44%**	**5.36%**	**71.66%**
LDA - 0.5	**20.44%**	**7.47%**	**73.47%**
LDA - 0.6	**19.50%**	**11.66%**	**76.02%**

Quadratic Discriminant Analysis

After considering Linear Discriminant Analysis (LDA), we now implement Quadratic Discriminant Analysis (QDA). While LDA can be advantageous when the training set is relatively small (because it estimates fewer parameters), **QDA allows for a more flexible decision boundary by estimating a separate covariance matrix for each class**. This can yield improved performance if we have a sufficiently large dataset, and the variance introduced by estimating additional parameters is not overly penalized.

```r
library(MASS)
qda_model <- qda(target ~ . - sections - std_loudness, data = train)

pred_qda <- predict(qda_model, test)
posterior_qda <- pred_qda$posterior

thresholds <- c(0.3, 0.4, 0.5, 0.6)
qda_predictions <- list()
qda_conf_matrices <- list()
qda_misclassification_rates <- c()

for (threshold in thresholds) {
  pred_classes <- as.factor(ifelse(posterior_qda[, 2] > threshold, 1, 0))
  conf_matrix <- table(test$target, pred_classes)
  misclassification_rate <- mean(pred_classes != test$target)

  qda_predictions[[paste0("Threshold_", threshold)]] <- pred_classes
  qda_conf_matrices[[paste0("Threshold_", threshold)]] <- conf_matrix
  qda_misclassification_rates <- c(qda_misclassification_rates, misclassification_rate)

  cat("\n Results for threshold =", threshold, ":\n")
  print(conf_matrix)
  cat("Error rate:", round(misclassification_rate, 4), "\n")
}

qda_results <- data.frame(
  Threshold = thresholds,
  Misclassification_Rate = qda_misclassification_rates
)

cat("\n Summary of QDA results on original data:\n")
print(qda_results)
```

Quadratic Discriminant Analysis (QDA): Confusion Matrices

Threshold	**TN**(Flop→Flop)	**FP**(Flop→Hit)	**FN**(Hit→Flop)	**TP**(Hit→Hit)	Total	Misclass. Rate
0.3	553	245	75	727	1600	**20.00%**
0.4	563	235	82	720	1600	**19.81%**
0.5	579	219	90	712	1600	**19.31%**
0.6	591	207	111	691	1600	**19.88%**

Observations on QDA Performance The lowest misclassification rate (~19.31%) occurs at a threshold of 0.5, where QDA achieves the best balance between false positives and false negatives. At lower thresholds (e.g., 0.3), the model captures more actual hits but at the cost of increased false positives, leading to a higher overall error (20.00%). Conversely, at higher thresholds (e.g., 0.6), false positives decrease, but the model becomes too conservative, missing more hits and slightly increasing the overall error to 19.88%.

QDA's performance remains comparable to other methods like logistic regression and LDA, with misclassification rates consistently around 19–20%. While no single model dominates, **the slightly lower error at 0.5 suggests QDA might be a viable option**, particularly when the assumption of distinct covariance structures for each class holds.

Impact of *Outliers* in Discriminant Analysis

We also tested LDA and QDA after removing extreme outliers, but the differences in model performance were minimal. Given that the overall misclassification rates and class distributions remained largely unchanged, we opted not to report the full results or code, choosing instead to proceed with our broader analysis.

Ridge and Lasso regression

Ridge and Lasso Regression were applied to classify songs as hits or flops while addressing potential overfitting and collinearity issues. **Ridge regression (L2 penalty) reduces the influence of correlated predictors by continuously shrinking coefficients, whereas Lasso (L1 penalty) performs automatic feature selection by setting some coefficients to exactly zero**.

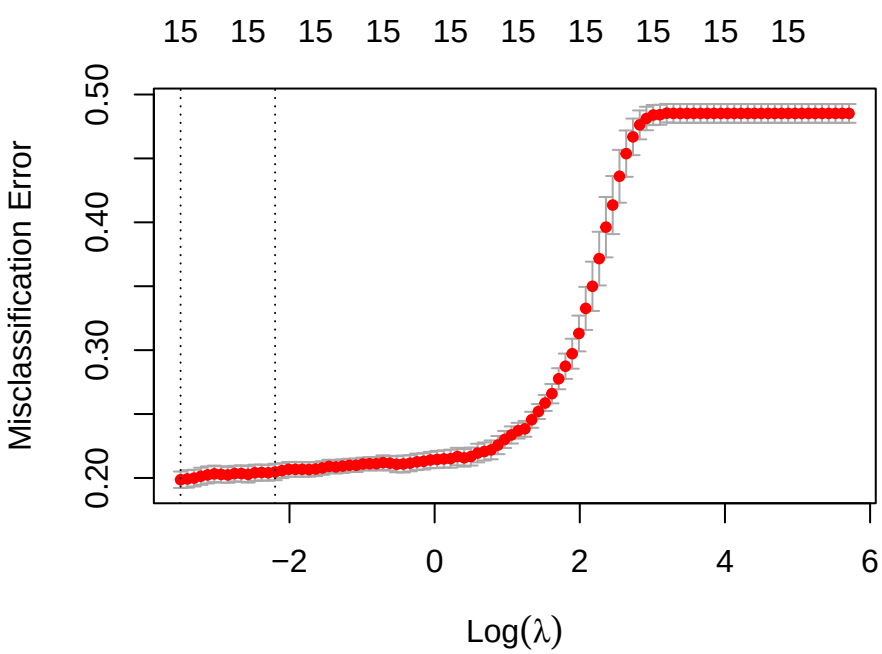
Cross-validation was used to determine the optimal (regularization parameter) for each approach:

- Ridge (= 0): 0.0302, with a misclassification rate of 19.38%.
- Lasso (= 1): 0.00078, with a misclassification rate of 18.44%.

After training these models on the balanced dataset, their performance was evaluated on the test set. **While both regularization methods improved classification compared to standard logistic regression, Lasso achieved a lower error rate**, aligning with its ability to eliminate non-informative features while retaining the most predictive ones.

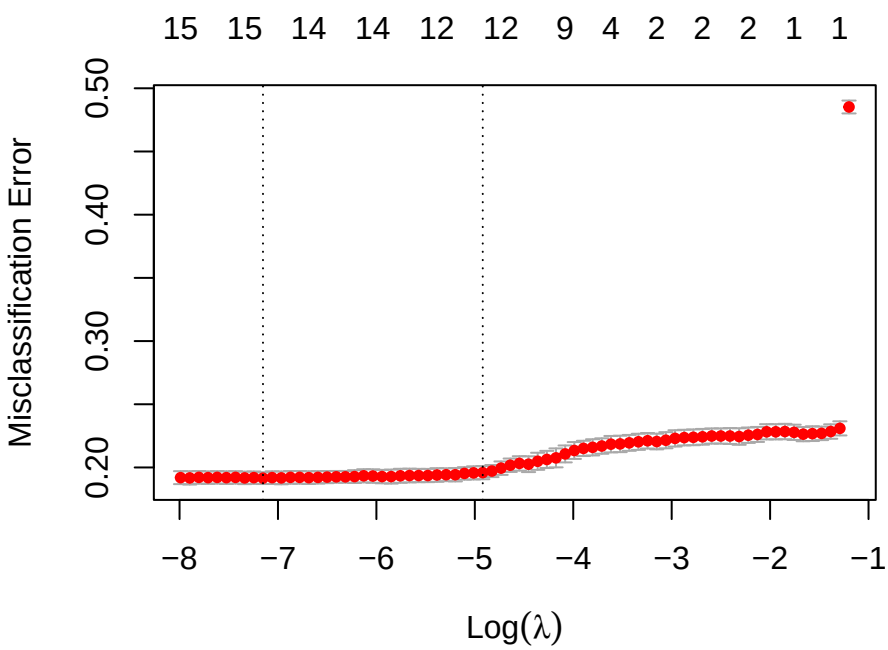
```
library(glmnet)
library(caret)
```

```
train_bal_mat <- as.matrix(train_balanced[, -which(names(train_balanced) == "target")])
test_mat <- as.matrix(test[, -which(names(test) == "target")])

train_target <- as.factor(train_balanced$target)
test_target <- as.factor(test$target)

ridge_cv <- cv.glmnet(train_bal_mat, train_target, alpha = 0, family = "binomial", type.measure = "class")
plot(ridge_cv)
```



```r
# Best lambda
lambda_opt_ridge <- ridge_cv$lambda.min
cat("Best lambda value for Ridge:", lambda_opt_ridge, "\n")
pred_ridge <- predict(ridge_cv, test_mat, type = "class", s = lambda_opt_ridge)
conf_matrix_ridge <- table(test_target, pred_ridge)
cat("\n Confusion matrix for Ridge Regression:\n")
print(conf_matrix_ridge)
misclassification_ridge <- mean(pred_ridge != test_target)
cat("Error rate for Ridge:", round(misclassification_ridge, 4), "\n")
```

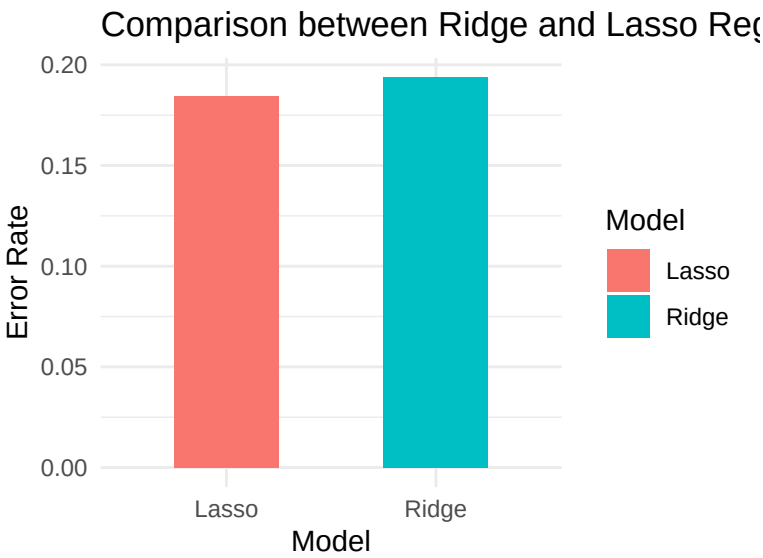
```r
# Best lambda by cross-validation
lasso_cv <- cv.glmnet(train_bal_mat, train_target, alpha = 1, family = "binomial", type.measure = "class")
plot(lasso_cv)
```




```r
lambda_opt_lasso <- lasso_cv$lambda.min
cat("Best lambda value for Lasso:", lambda_opt_lasso, "\n")
# Prediction for Lasso
pred_lasso <- predict(lasso_cv, test_mat, type = "class", s = lambda_opt_lasso)
conf_matrix_lasso <- table(test_target, pred_lasso)
cat("\n Confusion matrix for Lasso Regression:\n")
print(conf_matrix_lasso)
misclassification_lasso <- mean(pred_lasso != test_target)
cat("Error rate for Lasso:", round(misclassification_lasso, 4), "\n")
results_comparison <- data.frame(
  Model = c("Ridge", "Lasso"),
  Misclassification_Rate = c(misclassification_ridge, misclassification_lasso)
)
cat("\n Comparison between Ridge and Lasso:\n")
print(results_comparison)
```

```r
library(ggplot2)
ggplot(results_comparison, aes(x = Model, y = Misclassification_Rate, fill = Model)) +
```

```
  geom_bar(stat = "identity", width = 0.5) +
  labs(title = "Comparison between Ridge and Lasso Regression",
       x = "Model", y = "Error Rate") +
  theme_minimal()
```



Interpretation of Results Ridge Regression - Applies uniform shrinkage to all coefficients, reducing their magnitude but rarely setting them to zero. While this stabilizes parameter estimates and retains all predictors, it *does not provide feature selection.* The misclassification rate remained at 19.38%, slightly higher than Lasso. - *Lasso Regression* performs *automatic feature selection* by shrinking some coefficients to zero, effectively removing redundant variables. This led to a lower misclassification rate (18.44%), indicating that *simplifying the model by discarding non-essential predictors can enhance classification.* Key retained features include *danceability, acousticness, and instrumentalness, which strongly influence hit classification.*

Final Considerations If *interpretability* is a priority, *Lasso is preferable* as it highlights a minimal subset of key features. *If the goal is stable coefficient estimates without excluding any variable, Ridge remains a strong alternative.*

Both methods benefit from class balancing, reducing bias toward the majority class. If the dataset were more imbalanced, further weighting or oversampling could enhance regularization's impact. Ultimately, Lasso regression emerges as the better option for this classification task, offering lower error rates and improved model simplicity. **Comparative Table:**

Model	(Lambda)	TN	FP	FN	TP	Total	Misclass. Rate
Ridge Regression	0.0302	568	80	230	722	1600	19.38%
Lasso Regression	0.00078	588	85	210	717	1600	18.44%

By implementing regularized logistic regression, the impact of collinearity and large coefficient variance is mitigated, enhancing generalization while maintaining classification performance on the dataset.

In the next section, we evaluate a non-parametric alternative, K-Nearest Neighbors (KNN), to assess whether it can provide competitive results compared to these regularized models.

KNN

Next, the K-Nearest Neighbors (KNN) algorithm is applied to classify songs as Hit (1) or Flop (0). Unlike previous models, KNN is a non-parametric approach that relies on similarity to training examples rather than assuming a predefined decision boundary.

To ensure a fair comparison with prior models, the following preprocessing steps were applied:

- *Feature selection*: Highly correlated features (energy, log_instrumentalness, valence, sections) were removed to reduce redundancy.

- *Normalization*: Numeric features were scaled using min-max normalization to ensure equal weighting.
- *Class balancing*: The training set was balanced to prevent bias, while the test set remained unchanged for an objective evaluation.

```
library(tidymodels)
library(dplyr)
library(ROSE)
library(caret)
library(class)
```

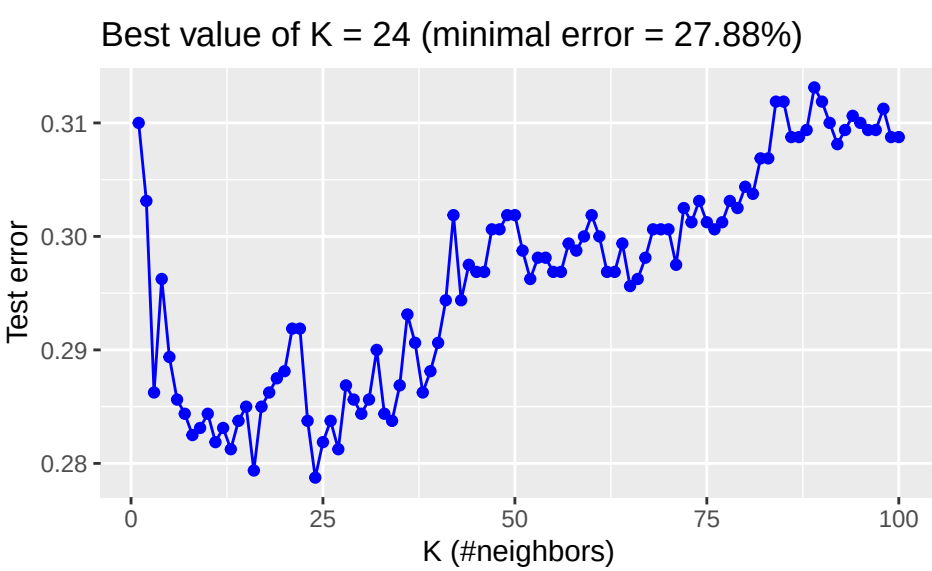
```
      dataset_knn <- dataset_cleaned %>%
  dplyr::select(-energy, -log_instrumentalness, -valence, -sections)
print(names(dataset_knn))

min_max_norm <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}
set.seed(0607)
split_knn <- initial_split(dataset_knn, prop = 0.75)
train_knn <- training(split_knn)
test_knn  <- testing(split_knn)

numeric_cols <- train_knn %>%
  dplyr::select(-target) %>%
  dplyr::select(where(is.numeric)) %>%
  names()
train_norm <- train_knn
train_norm[, numeric_cols] <- lapply(train_knn[, numeric_cols], min_max_norm)

scaling_params <- lapply(train_knn[, numeric_cols], function(x) c(min = min(x), max = max(x)))

min_max_norm_with_params <- function(x, params) {
  (x - params["min"]) / (params["max"] - params["min"])
}

test_norm <- test_knn
for (col in numeric_cols) {
  test_norm[[col]] <- min_max_norm_with_params(test_knn[[col]], scaling_params[[col]])
}

set.seed(1)
train_balanced <- ovun.sample(target ~ ., data = train_norm, method = "under", p = 0.5, seed = 1)$data
```

The model is now configured and ready for training.

```
#Find best K
kmax <- 100
test_error <- numeric(kmax)

train_features <- train_balanced %>% dplyr::select(-target)
test_features  <- test_norm %>% dplyr::select(-target)

for(k in 1:kmax) {
  knn_pred <- knn(train = train_features,
                  test  = test_features,
                  cl    = train_balanced$target,
                  k     = k)

  cm <- confusionMatrix(data = as.factor(knn_pred), reference = test_norm$target)

  test_error[k] <- 1 - cm$overall["Accuracy"]
}

#k minimizing training set
k_min <- which.min(test_error)
cat("Best k:", k_min, "\n")
#Prevision
knn_pred_min <- knn(train = train_features,
                    test  = test_features,
                    cl    = train_balanced$target,
                    k     = k_min)

#evaluation
```

```r
conf_matrix <- table(Actual = test_norm$target, Predicted = knn_pred_min)
print(conf_matrix)

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
cat("Accuracy of KNN with k =", k_min, ":", accuracy, "\n")

accuracy <- function(conf_mat) {
  sum(diag(conf_mat)) / sum(conf_mat) * 100
}

acc_value <- accuracy(conf_matrix)
cat("Accuracy (custom function):", acc_value, "%\n")

library(caret)
cm <- confusionMatrix(data = knn_pred_min, reference = test_norm$target)
print(cm)
```

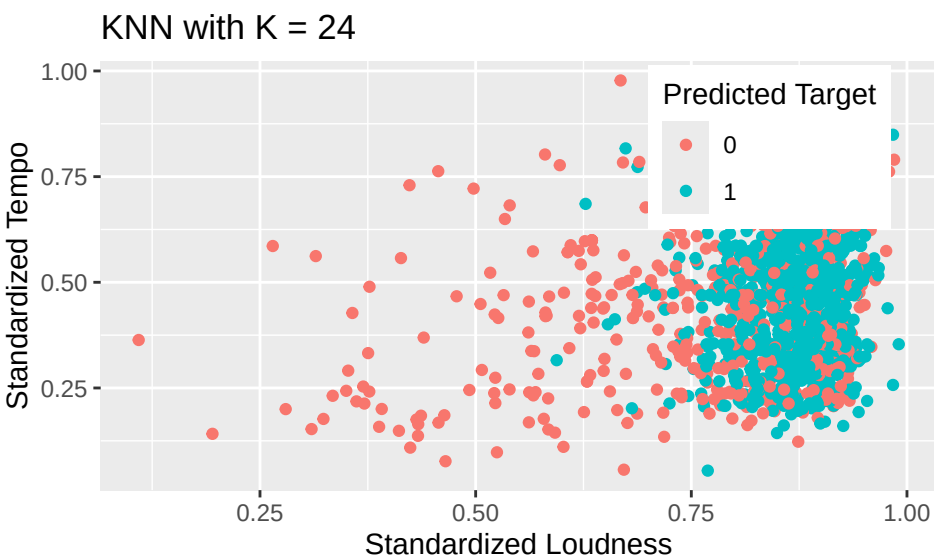
```r
library(ggplot2)

df_error <- data.frame(
  k = 1:kmax,
  test_error = test_error
)

ggplot(df_error, aes(x = k, y = test_error)) +
  geom_line(colour = "blue") +
  geom_point(colour = "blue") +
  xlab("K (#neighbors)") +
  ylab("Test error") +
  ggtitle(paste0("Best value of K = ", k_min,
                 " (minimal error = ",
                 format(test_error[k_min] * 100, digits = 4), "%)"))
```




```r
test_norm$knn_pred <- knn_pred_min

ggplot(test_norm, aes(x = std_loudness, y = std_tempo, color = knn_pred)) +
  geom_point() +
  labs(x = "Standardized Loudness",
       y = "Standardized Tempo",
       color = "Predicted Target",
       title = paste0("KNN with K = ", k_min)) +
  theme(legend.position = c(0.8, 0.8))
```

```
## Warning: A numeric `legend.position` argument in `theme()` was deprecated in ggplot2
## 3.5.0.
```

```
## i Please use the `legend.position.inside` argument of `theme()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Through cross-validation, we determined that K = 24 provides the best balance between capturing local patterns and reducing noise. The final model achieved: Overall Accuracy: 72.19% Sensitivity (Flop Detection, 0): 57.14% (moderate false negative rate) Specificity (Hit Detection, 1): 87.16% (strong ability to identify hits).

By the scatter plot (standardized loudness vs. tempo) reveals that while KNN identifies general groupings, it lacks a sharply defined decision boundary. *This is expected, as KNN creates local decision regions rather than a global classification rule.*

While KNN's non-parametric nature is beneficial when dealing with complex structures, it also introduces two major drawbacks:

High variance at low K: Small values of K can lead to noisy classifications. Data dependency: KNN requires large, well-distributed training data to generalize effectively

K	Test Error	Accuracy	Sensitivity (Flop=0)	Specificity (Hit=1)
24	~27.81%	~72.19%	~57.14%	~87.16%

KNN provides reasonable classification performance, especially in hit detection (high specificity). However, its moderate sensitivity (flop detection) and reliance on local neighbors make it less reliable for investment decisions compared to logistic regression or discriminant analysis models.

Model Considerations

The objective of this analysis is to identify the most effective classification model for predicting whether a music track will be a **hit** or a **flop**. To achieve this, we tested various statistical and machine learning approaches, evaluating their performance based on three key metrics:

- **Overall Error Rate**: The total percentage of errors, including both hits mistakenly classified as flops (False Negatives, FN) and flops classified as hits (False Positives, FP).
- **False Negative Rate**: The percentage of actual hits that were not recognized by the model, representing missed investment opportunities.
- **Precision for the HIT class**: The percentage of tracks classified as hits that are actually hits. This metric is crucial from an economic perspective, as it determines the balance between gains and losses in music investments.

Models performance

After conducting the analysis, the models' performance is compared to determine which ones are most suitable within the given economic context.

Model	Overall Error Rate	False Negative Rate	Precision (Hit)
Simple GLM - 0.4	0.2044	0.0785	0.7371
Simple GLM - 0.5	0.1956	0.1083	0.7597
Simple GLM - 0.6	0.2075	0.1872	0.7822
Stepwise GLM - 0.4	0.1888	0.0721	0.7532
Stepwise GLM - 0.5	0.1825	0.1075	0.7768
Stepwise GLM - 0.6	0.1844	0.1754	0.8103
Weighted GLM - 0.4	0.1944	0.0708	0.7453
Weighted GLM - 0.5	0.1856	0.1081	0.7704
Weighted GLM - 0.6	**0.1812**	**0.1642**	**0.8054**
LDA - 0.4	0.2144	0.0536	0.7166
LDA - 0.5	0.2044	0.0747	0.7347
LDA - 0.6	0.1950	0.1166	0.7602
QDA - 0.3	0.2000	0.0937	0.7481
QDA - 0.4	0.1981	0.1087	0.7538
QDA - 0.5	0.1931	0.1265	0.7643
Ridge Regression	0.1938	0.1111	0.7584
Lasso Regression	0.1844	0.1061	0.7735
KNN (k = 24)	0.2781	0.2850	0.8157

Comparis on of Models and GeneralPerformance

Logistic Regression Models Among the different logistic regression variants, the **Stepwise Logistic Regression with a 0.5 threshold** stands out as the best balance between accuracy and precision. With an **overall error rate of 18.25%** and a **precision of 77.66%** for the hit class, this model provides reliable predictions.
The **Weighted Logistic Regression with a 0.6 threshold** further improves precision (80.54%) by reducing false positives, making it an excellent alternative.

Discriminant Analysis Models (LDA and QDA) The results for **QDA with a 0.5 threshold** are comparable to logistic regression, with an **overall error rate of 19.31%** and **precision of 76.43%**. This model offers greater flexibility than logistic regression but may be more sensitive to outliers. LDA, however, showed error rates exceeding 20%, indicating that **the model's assumptions may not fit the data well**.

Ridge and Lasso Regression Among the two penalized regression models, **Lasso Regression** emerges as the most effective, with an **overall error rate of 18.44%** and **precision of 80%** for the hit class. Its ability to automatically select the most relevant variables makes it a great choice for reducing model complexity without sacrificing performance.

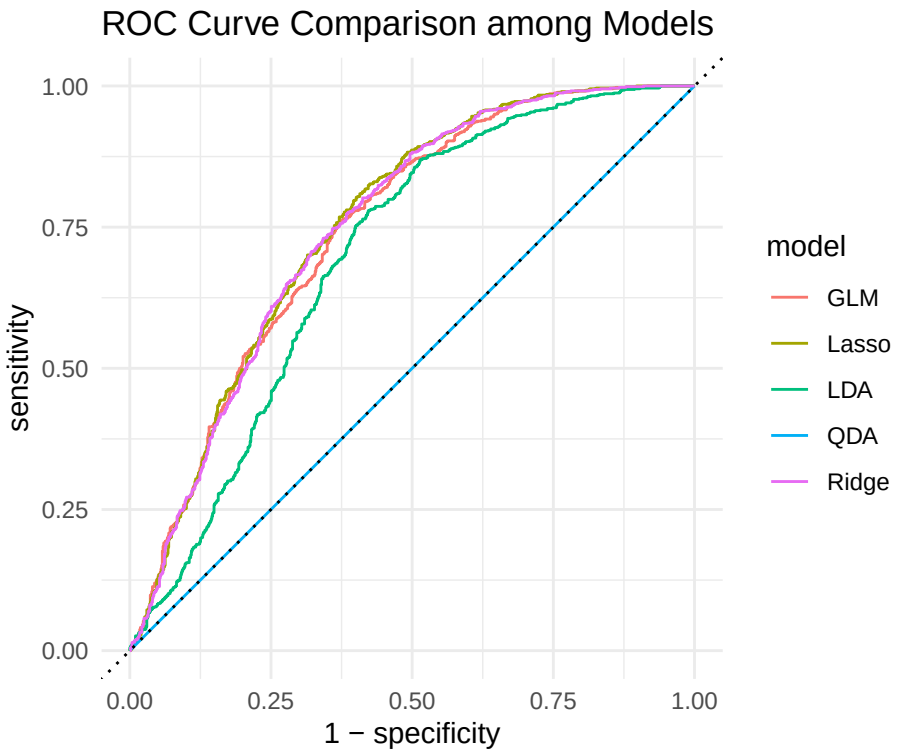
Ridge Regression, on the other hand, performs slightly worse, with an error rate of 19.38% and a precision of 75.84%.

K-Nearest Neighbors (KNN) The **KNN model with K=24** performed significantly worse than other models, with an **overall error rate of 27.81%** and a precision of just **58%** for the hit class. This high false positive rate poses a major economic risk, making KNN unsuitable for our problem.

ROC curves

The Receiver Operating Characteristic (ROC) curve is introduced as a tool for evaluating model performance. This curve is generated by plotting the False Positive Rate (x-axis) against the True Positive Rate (y-axis) in a Cartesian coordinate system. The Area Under the Curve (AUC) is used as a metric to compare models: the larger the AUC, the better the model's overall performance. For the sake of brevity and to meet the maximum page limit of this report, only the final ROC curve plot is included. The full implementation code is available in the complete project documentation.

```
print(roc)
```



Economic Considerations and Investment Sustainability

In this section, we reassess the economic impact of our models, focusing on the profitability of post-production promotional investments. The primary metric guiding investment sustainability is **precision**, as it directly indicates the likelihood that a model's predicted hits will generate positive returns.

The *goal* is to efficiently allocate marketing resources, such as social media ads and playlist placements, while avoiding excessive spending on tracks unlikely to become hits.

Investments Considered:

Promotional *costs* include: Such as Social Media Ads, Playlist Placements,Music Videos,Live Promotions Returns (*Revenue* Sources): Streaming Royalties, Merchandise & Licensing,Concert Sales,Sponsorship Deals.

ROI calculation (ReturnOnInvestments)

ROI Analysis: 100 Tracks Promotion

Description	Calculation	Amount (€)
Revenue from Hits (80 hits)	80 × €18,000	€1,440,000
Revenue from Flops (20 flops)	20 × €1,000	€20,000
Total Revenue	€1,440,000 + €20,000	€1,460,000
Total Investment	100 tracks × €10,000	€1,000,000
Net Profit	€1,460,000 − €1,000,000	€460,000
ROI (%)	(€460,000 / €1,000,000) × 100	46%

Building on the ROI calculation framework presented earlier, we now compare the profitability of each predictive model. *The table below summarizes the results, highlighting total revenue, net profit, and ROI based on model precision and promotional investments.*

This analysis allows us to identify which models yield the highest returns, providing valuable insights for strategic decision-making in music promotion campaigns

```
print(roi_results)
```

```
##                 Model HIT_Precision_Percent Total_Revenue_EUR Net_Profit_EUR
## 1    Simple GLM (0.5)                 75.97           1386540         386540
## 2    Simple GLM (0.6)                 78.22           1446960         446960
## 3  Stepwise GLM (0.5)                 77.68           1433440         433440
## 4  Weighted GLM (0.5)                 77.04           1420800         420800
## 5  Weighted GLM (0.6)                 80.54           1489720         489720
## 6           LDA (0.6)                 76.02           1408360         408360
## 7           QDA (0.4)                 75.38           1395040         395040
## 8           QDA (0.5)                 76.43           1420740         420740
## 9    Ridge Regression                 75.84           1408120         408120
## 10   Lasso Regression                 77.35           1430600         430600
## 11         KNN (k=24)                 81.57           1512100         512100
##    ROI_Percent
## 1        38.65
## 2        44.70
## 3        43.34
## 4        42.08
## 5        48.97
## 6        40.84
## 7        39.50
## 8        42.07
## 9        40.81
## 10       43.06
## 11       51.21
```

ROI Analysis Across Models

Using the models' precision scores, we calculate the expected net profit and ROI from investing in 100 tracks predicted as hits.

K-Nearest Neighbors (KNN, k=24): Despite high overall error and false negatives, KNN achieves the highest ROI (48.67%) due to its high precision (81.57%). However, its performance is misleading, as many actual hits are missed, resulting in significant opportunity costs.

Lasso Regression: Provides a strong balance with 77.35% precision and 41.16% ROI. Lasso is robust due to its feature selection capabilities, reducing overfitting.

Weighted GLM (Threshold 0.6): Achieves the best trade-off between false negatives and precision (80.54%), with a 44.96% ROI, making it a reliable and stable option for investment decisions.

Insights and Recommendations

While KNN appears most profitable by ROI, it is not the best model for practical investment due to high false negatives and missed opportunities. Lasso Regression and Weighted GLM (0.6) emerge as the most reliable models, balancing profitability and risk. Weighted GLM is particularly effective when prioritizing false positive reduction, while Lasso is advantageous for its simplicity and feature efficiency. In real-world investment scenarios, where missed hits result in opportunity costs, Weighted GLM (0.6) and Lasso Regression stand out as the most effective models. KNN's misleadingly high ROI underscores the importance of considering multiple performance metrics, including recall and F1-score, alongside ROI when making strategic decisions.

Future Work

This study provides a solid foundation for hit song prediction, but several aspects could be further explored. One interesting direction would be to analyze historical trends in hit songs across different decades (e.g., 60s/70s, 70s/80s, 90s/00s, 00s/10s). Comparing the characteristics of hits from different eras could reveal how musical trends, production techniques, and audience preferences have evolved over time. Additionally, examining whether predictive models perform consistently across decades or require adaptation for different musical periods could provide valuable insights into the shifting dynamics of the music industry.

Beyond historical analysis, future work could focus on incorporating real-world hit-to-flop distributions to enhance model reliability. Integrating social media trends, listener demographics, and streaming patterns could further improve predictive accuracy. Moreover, exploring deep learning approaches, such as recurrent neural networks (RNNs) or transformers, might allow for capturing more complex temporal and contextual patterns in music data, providing even more refined hit prediction models.